

pipeline

Lorys AGBATCHOSSOU · 3 de septembre de 2026



Etape 1 : Ingestion et nettoyage des données

SPY → marché actions américain QQQ → Nasdaq / tech / growth TLT → obligations américaines long terme GLD → or USO → pétrole

```
In [2]:
import yfinance as yf

tickers = ["SPY", "TLT", "GLD", "USO", "QQQ"]

data = yf.download(tickers, period="5y", auto_adjust=True)["Close"] #prix au close
data
```

Out [2]:
[*****100%*****] 5 of 5 completed

Ticker	GLD	QQQ	SPY	TLT	USO
Date					
2021-09-03	171.059998	370.309692	423.462555	124.220276	48.660000
2021-09-07	167.710007	370.833618	421.948395	123.172371	48.040001
2021-09-08	167.289993	369.542847	421.434448	124.010658	48.590000
2021-09-09	168.029999	368.271576	419.630524	125.511215	47.750000
2021-09-10	167.179993	365.476654	416.321930	124.404602	48.799999
...	...	...	...	...	...
2026-08-27	422.600006	721.109985	771.099976	82.812668	130.009995
2026-08-28	408.890015	716.429993	769.349976	82.563622	129.699997
2026-08-31	408.420013	716.760010	767.049988	82.204994	133.699997
2026-09-01	396.750000	707.640015	761.780029	81.870003	141.000000
2026-09-02	402.779999	709.239990	765.159973	81.949997	141.149994

1254 rows × 5 columns

```
In [3]:
#rendements journaliers
returns = data.pct_change().dropna() #on supprime la premiere ligne elle sera vide

#sauvegarde
returns.to_parquet("returns.parquet")

returns
```

Out [3]:

Ticker	GLD	QQQ	SPY	TLT	USO
Date					
2021-09-07	-0.019584	0.001415	-0.003576	-0.008436	-0.012741
2021-09-08	-0.002504	-0.003481	-0.001218	0.006806	0.011449
2021-09-09	0.004423	-0.003440	-0.004280	0.012100	-0.017288
2021-09-10	-0.005059	-0.007589	-0.007885	-0.008817	0.021990
2021-09-13	0.003350	-0.000691	0.002559	0.006065	0.013934
...	...	...	...	...	...
2026-08-27	0.003038	0.013692	0.006553	-0.002041	0.020887
2026-08-28	-0.032442	-0.006490	-0.002269	-0.003007	-0.002384
2026-08-31	-0.001149	0.000461	-0.002990	-0.004344	0.030840
2026-09-01	-0.028574	-0.012724	-0.006870	-0.004075	0.054600
2026-09-02	0.015198	0.002261	0.004437	0.000977	0.001064

1253 rows × 5 columns

Etape 2 : Analyse exploratoire EDA

```

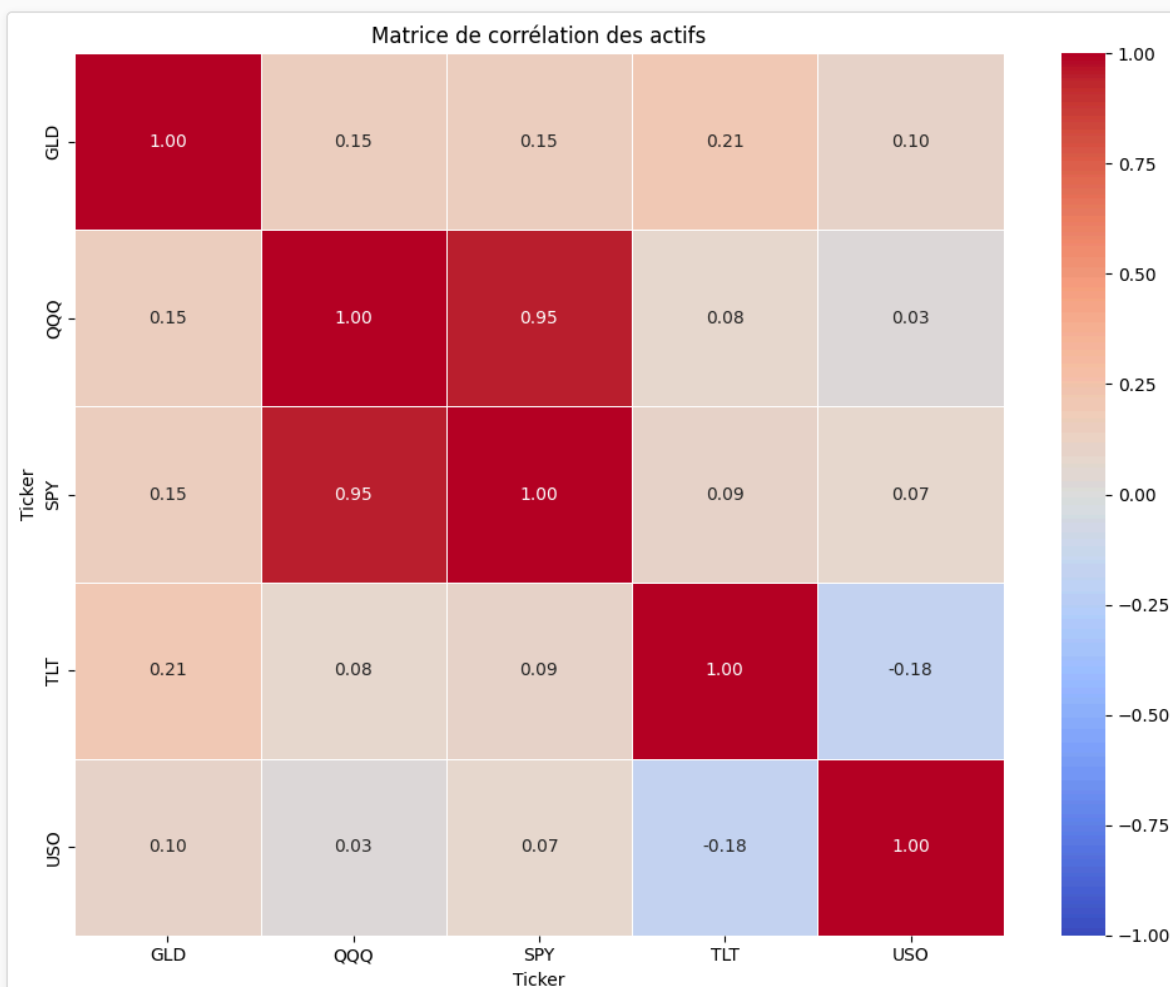
In [4]:
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

corr = returns.corr()

#matrice de correlation
plt.figure(figsize=(10, 8))
sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    linewidths=0.5,
    fmt=".2f"
)
plt.title("Matrice de corrélation des actifs")
plt.tight_layout()
plt.show()

```

Out [4]:



On remarque qu'il n'y a presque pas de corrélation entre les actifs (toute proche de 0), quoique les actions US et actions TECH sont évoluent fortement dans le même sens (0.95 proche de 1). Ce qui me semble logique car le marché des actions US est de loin majoritairement dominé par les grosses boîtes de TECH

(NVIDIA, Apple, Microsoft...)

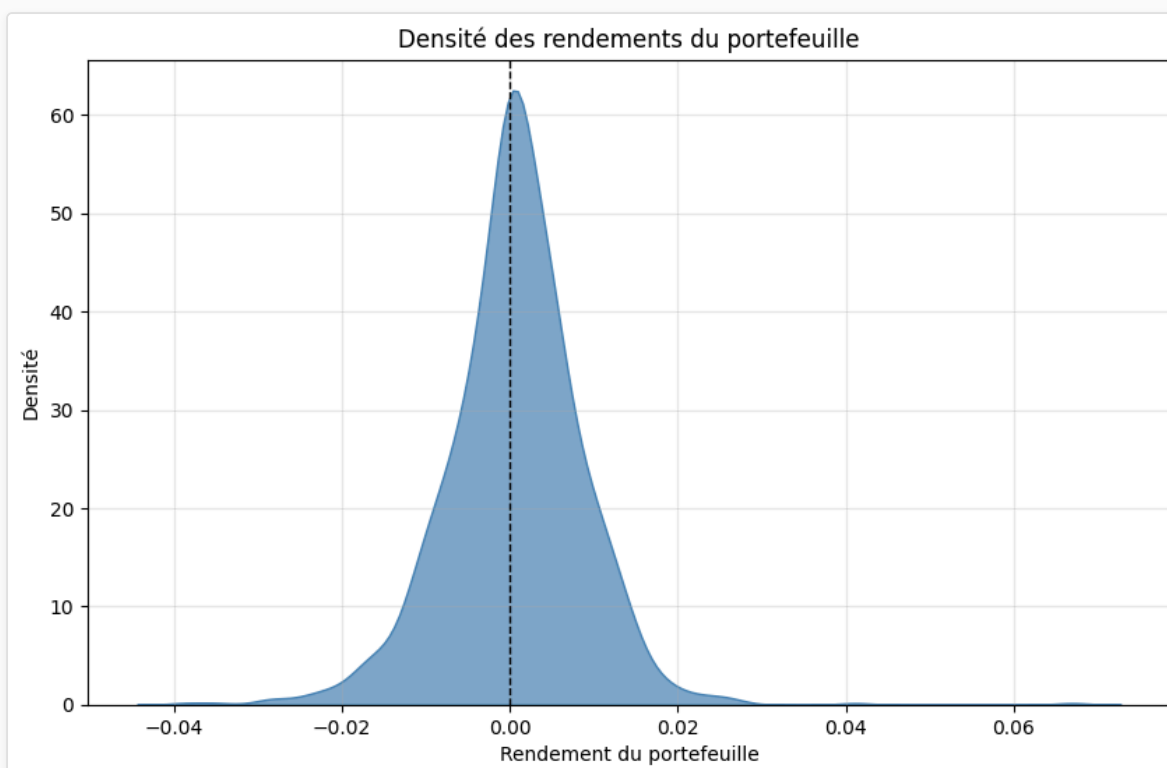
```
In [5]:
#courbe de densité des rendements du portefeuille
import numpy as np

# Portefeuille équipondéré 20%
poids = np.array([1/5, 1/5, 1/5, 1/5, 1/5])

#rendement du portefeuille = somme pondérée des rendements
portfolio_return = returns.dot(poids)

# Courbe de densité
plt.figure(figsize=(10, 6))
sns.kdeplot(portfolio_return, fill="True", color="steelblue", alpha=0.7)
plt.axvline(0, color="black", linestyle="--", linewidth=1)
plt.title("Densité des rendements du portefeuille")
plt.xlabel("Rendement du portefeuille")
plt.ylabel("Densité")
plt.grid(True, alpha=0.3)
plt.show()
```

Out [5]:



```
In [6]:
# superposition avec une loi normale de meme moyenne/ecart-type
from scipy import stats

#parametres de la loi normale
mu = portfolio_return.mean()
sigma = portfolio_return.std(ddof=1)

# Grille pour la courbe de densité normale
x = np.linspace(portfolio_return.min(), portfolio_return.max(), 500)

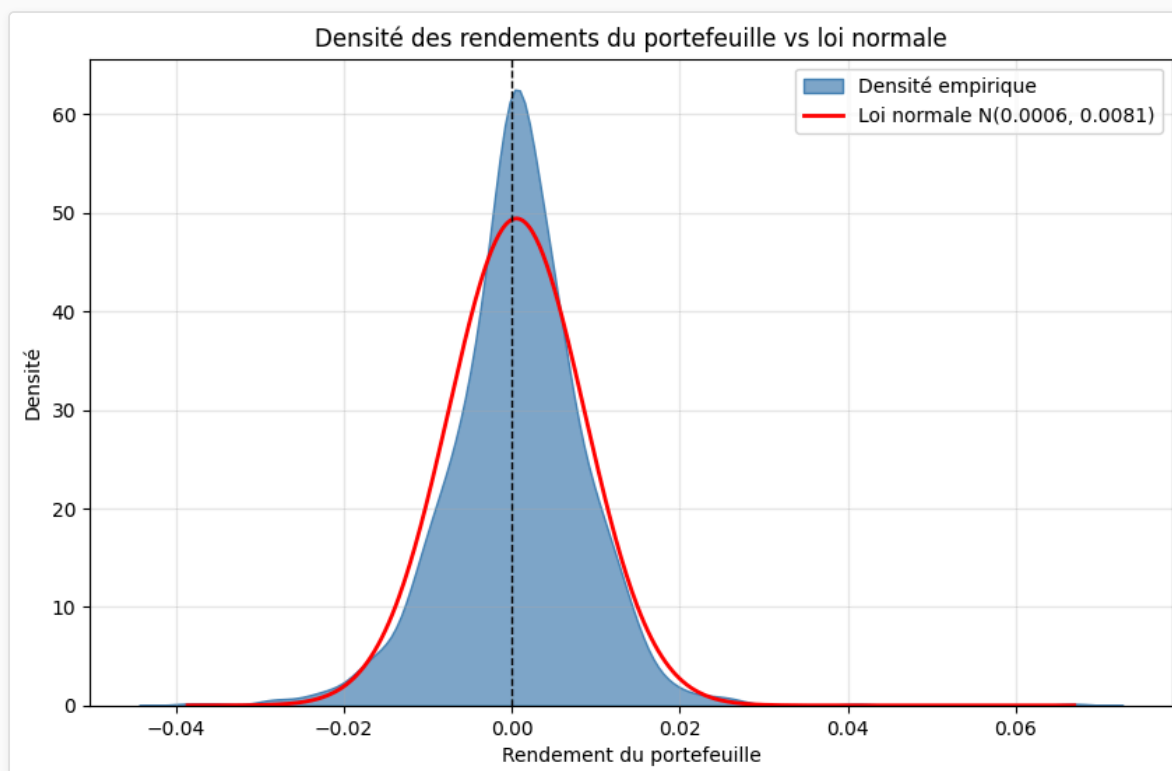
# Graphique
plt.figure(figsize=(10, 6))

# Densité empirique du portefeuille
sns.kdeplot(portfolio_return, fill=True, color="steelblue", alpha=0.7, label="Densité empirique")
```

```
# Loi normale de même moyenne / écart-type
plt.plot(
    x,
    stats.norm.pdf(x, loc=mu, scale=sigma),
    color="red",
    linewidth=2,
    label=f"Loi normale N({mu:.4f}, {sigma:.4f})"
)

plt.axvline(0, color="black", linestyle="--", linewidth=1)
plt.title("Densité des rendements du portefeuille vs loi normale")
plt.xlabel("Rendement du portefeuille")
plt.ylabel("Densité")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Out [6]:



Analyse de la distribution des rendements

Le pic de la densité des rendements du portefeuille est 12% plus élevé que le pic d'une loi normale de même écart-type et de même moyenne. De plus, la densité des rendements du portefeuille présente des queues légèrement plus épaisses que la loi normale.

In [7]:

```
# Skewness (asymétrie)
skewness = stats.skew(portfolio_return)

# Kurtosis (excès de kurtosis, par défaut dans scipy = Fisher)
kurtosis = stats.kurtosis(portfolio_return)

print(f"Skewness du portefeuille : {skewness:.6f}")
print(f"Kurtosis du portefeuille : {kurtosis:.6f}")
```

Out [7]:

```
Skewness du portefeuille : 0.195870
Kurtosis du portefeuille : 5.069037
```

Le kurtosis est de 5,072775 soit 2 fois plus qu'une loi normale (généralement 3). Ce qui confirme les observations plus haut par rapports aux fat tails (les queues plus épaisses) et au pic central plus haut. Il en ressort que le portfolio est plutôt stable au quotidien avec des potentiels chocs extrêmes positifs comme négatifs plus important que ce que prévoit la normale. La skewness de 0.197325 indique qu'on devrait s'attendre plutôt à des chocs positifs.

Calcul de la VaR (3 méthodes) Pour un horizon 1 jour, confiance 95% et 99% :

```
In [8]:
#VaR paramétrique
from scipy.stats import norm
#on suppose que les rendements du portfolio suivent une Loi normale
# parametres : moyenne mu, quantile z, et volatilité sigma (ecart-type)

def calcul_var(returns_, confiance):
    mu = np.mean(returns_)
    z = norm.ppf(1 - confiance)
    sigma = np.std(returns_, ddof = 1)
    var = -(mu + z*sigma)
    return var

print("VaR paramétrique confiance 95 % : ", calcul_var(portfolio_return, 0.95))
print("VaR paramétrique confiance 99 % : ", calcul_var(portfolio_return, 0.99))
```

```
Out [8]:
VaR paramétrique confiance 95 % :  0.012727613699355593
VaR paramétrique confiance 99 % :  0.018229036102201505
```

```
In [9]:
#var historique
# Le quantile empirique des rendements passés (pas d'hypothèse de distribution).
print(f"VaR historique confiance 95 % : {np.quantile(portfolio_return, 0.05)}" )
print(f"VaR historique confiance 99 % : {np.quantile(portfolio_return, 0.01)}" )
```

```
Out [9]:
VaR historique confiance 95 % : -0.012306760637989545
VaR historique confiance 99 % : -0.020830181128363878
```

```
In [10]:
#var Monte Carlo
# Paramètres de simulation
n_scenarios = 10_000
confiances = [0.95, 0.99]

# Moyenne et covariance des rendements des actifs
mu_actifs = returns.mean().values #pas le rendement du portefeuille mais celui des actifs
cov_actifs = returns.cov().values

# Simulation des rendements des actifs
simulations_actifs = np.random.multivariate_normal(
    mean=mu_actifs,
    cov=cov_actifs,
    size=n_scenarios
)

# Rendement simulé du portefeuille
simulations_portfolio = simulations_actifs.dot(poids)

for confiance in confiances:
    quantile = np.quantile(simulations_portfolio, 1 - confiance)
    var = -quantile

print(
    f"VaR Monte Carlo à {confiance:.0%} : "
    f"{var: }"
)
```

```
Out [10]:
VaR Monte Carlo à 99% :  0.01853492499100011
```

Niveau de confiance	VaR paramétrique	VaR historique	VaR Monte Carlo
95%	1.2723%	1.2299%	1.2846%
99%	1.8222%	2.0823%	1.8951%

La VaR à 99% donnée par la méthode historique est supérieure à celles données par les autres méthodes ($1.82\% < 1.85\% < 2.08\%$).

Il en ressort que le portfolio fait des pertes plus importantes que ce qui est attendu théoriquement. Les méthodes de calcul paramétrique et de Monte Carlo masquent la réalité car elles ne prennent pas en compte la tension réelle du marché.

Rappelons que la méthode paramétrique suppose une distribution normale des rendements, la méthode Monte Carlo simule 10000 scénarios aléatoires possibles mais qui peuvent ne pas refléter des comportements réels du portfolio.

Nous allons backtester la VaR.

Etape 4 : Backtesting de la VaR

backtest niveau de confiance 99%

```
In [11]:

"""
Compte les rendements inférieurs au seuil de perte de chaque VaR.
niveau de confiance 99%
Une exception correspond à : rendement réel < -VaR.
"""

VAR_PARAMETRIQUE_99 = 0.012723162022099745
VAR_HISTORIQUE_99 = 0.012299370663057089
VAR_MONTE_CARLO_99 = 0.018951352166484754

def calculer_exceptions(var, portfolio_return):
    return (portfolio_return < -var).sum()

print(f"{calculer_exceptions(VAR_PARAMETRIQUE_99, portfolio_return)} exceptions pour la méthode paramétrique à 99% de confiance")

print(f"{calculer_exceptions(VAR_HISTORIQUE_99, portfolio_return)} exceptions pour la méthode historique à 99% de confiance")

print(f"{calculer_exceptions(VAR_MONTE_CARLO_99, portfolio_return)} exceptions pour la méthode Monte Carlo à 99% de confiance")

Out [11]:
58 exceptions pour la méthode paramétrique à 99% de confiance
63 exceptions pour la méthode historique à 99% de confiance
17 exceptions pour la méthode Monte Carlo à 99% de confiance
```

backtest niveau de confiance 95%

```
In [12]:

"""
Compte les rendements inférieurs au seuil de perte de chaque VaR.
niveau de confiance 95%
Une exception correspond à : rendement réel < -VaR.
"""

VAR_PARAMETRIQUE_95 = 0.01822223008864799
VAR_HISTORIQUE_95 = 0.020823241379280686
VAR_MONTE_CARLO_95 = 0.01284682048312342

def calculer_exceptions(var, portfolio_return):
    return (portfolio_return < -var).sum()

print(f"{calculer_exceptions(VAR_PARAMETRIQUE_95, portfolio_return)} exceptions pour la méthode paramétrique à 95% de confiance")

print(f"{calculer_exceptions(VAR_HISTORIQUE_95, portfolio_return)} exceptions pour la méthode historique à 95% de confiance")

print(f"{calculer_exceptions(VAR_MONTE_CARLO_95, portfolio_return)} exceptions pour la méthode Monte Carlo à 95% de confiance")

Out [12]:
```

19 exceptions pour la méthode paramétrique à 95% de confiance
 13 exceptions pour la méthode historique à 95% de confiance
 56 exceptions pour la méthode Monte Carlo à 95% de confiance

Pour les 1254 jours, on doit s'attendre environ à 63 jours d'exceptions pour la VaR à 95% et environ 13 jours d'exceptions pour la VaR à 99%.

Pour la VaR 99%, les méthodes paramétrique et historique ont des jours d'exceptions presque 6 fois supérieures à ce qui est théoriquement attendu. Ces modèles de VaR sous-estiment de loin le risque réel. C'est ce que laissait présager les fat tails remarquées dans la distribution des rendements du portefeuille. Ce dernier est bien plus risqué qu'il n'en a l'air. Seul le modèle de Monte Carlo se rapproche de la réalité théorique à 5 jours d'exception près.

Pour la VaR 95%, les méthodes paramétrique et historique sur-estiment le risque réel. Le modèle Monte Carlo se rapproche un peu plus de la réalité théorique à 7 jours près.

Il ressort que la méthode de Monte Carlo serait celui qui permet d'évaluer le plus fidèlement possible la VaR. A vérifier avec un test de KUPIEC.

TEST DE KUPIEC

```
In [13]:
# Test de Kupiec : test de couverture inconditionnelle
#
# H0 : La fréquence observée des exceptions correspond à la fréquence théorique
#       attendue, soit 1 - niveau_de_confiance.
# H1 : La fréquence observée des exceptions est différente.
#
# Une exception correspond à un rendement réel inférieur à -VaR.
# La statistique LR suit asymptotiquement une loi du Chi-deux à 1 degré de Liberté.

def test_kupiec(var, rendements, niveau_confiance, alpha_test=0.05):
    n_observations = len(rendements)
    n_exceptions = int((rendements < -var).sum())

    probabilite_theorique = 1 - niveau_confiance
    probabilite_obseree = n_exceptions / n_observations

    # Log-vraisemblance binomiale, avec gestion des cas limites
    def log_vraisemblance(k, n, p):
        resultat = 0
        if k > 0:
            resultat += k * np.log(p)
        if n - k > 0:
            resultat += (n - k) * np.log(1 - p)
        return resultat

    # Vraisemblance sous H0 et sous H1
    log_L0 = log_vraisemblance(
        n_exceptions, n_observations, probabilite_theorique
    )
    log_L1 = log_vraisemblance(
        n_exceptions, n_observations, probabilite_obseree
    )

    # Statistique du rapport de vraisemblance de Kupiec
    LR_uc = 2 * (log_L1 - log_L0)

    # P-value selon une loi du Chi-deux à 1 degré de Liberté
    p_value = 1 - stats.chi2.cdf(LR_uc, df=1)

    return {
        "VaR": var,
        "Confiance": niveau_confiance,
        "Observations": n_observations,
        "Exceptions": n_exceptions,
        "Exceptions attendues": n_observations * probabilite_theorique,
        "Fréquence observée": probabilite_obseree,
        "Statistique LR": LR_uc,
        "p-value": p_value,
        "Décision": (
            "Rejet de H0"
            if p_value < alpha_test
        )
    }
```

```

        else "Non-rejet de H0"
    )
}

# VaR à 95 %
resultats_95 = [
    test_kupiec(VAR_PARAMETRIQUE_95, portfolio_return, 0.95),
    test_kupiec(VAR_HISTORIQUE_95, portfolio_return, 0.95),
    test_kupiec(VAR_MONTE_CARLO_95, portfolio_return, 0.95)
]

# VaR à 99 %
resultats_99 = [
    test_kupiec(VAR_PARAMETRIQUE_99, portfolio_return, 0.99),
    test_kupiec(VAR_HISTORIQUE_99, portfolio_return, 0.99),
    test_kupiec(VAR_MONTE_CARLO_99, portfolio_return, 0.99)
]

# Présentation des résultats
colonnes = [
    "VaR", "Confiance", "Observations", "Exceptions",
    "Exceptions attendues", "Fréquence observée",
    "Statistique LR", "p-value", "Décision"
]

resultats_95_df = pd.DataFrame(resultats_95, index=[
    "Paramétrique", "Historique", "Monte Carlo"
])[colonnes]

resultats_99_df = pd.DataFrame(resultats_99, index=[
    "Paramétrique", "Historique", "Monte Carlo"
])[colonnes]

print("Test de Kupiec à 95 %")
display(resultats_95_df)

print("Test de Kupiec à 99 %")
display(resultats_99_df)

```

Out [13]:

Test de Kupiec à 95 %

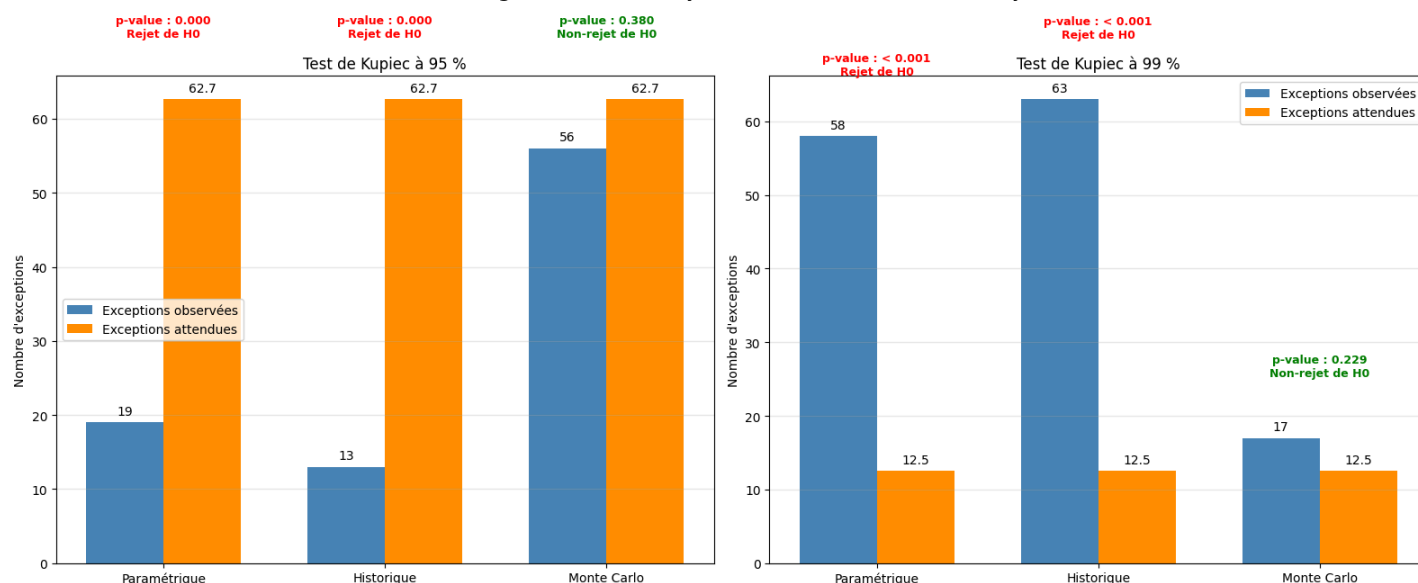
	VaR	Confiance	Observations	Exceptions	Exceptions attendues	Fréquence observée	Statistique LR	p-value	Décision
Paramétrique	0.018222	0.95	1253	19	62.65	0.015164	43.542688	4.148115e-11	Rejet de H0
Historique	0.020823	0.95	1253	13	62.65	0.010375	60.454743	7.549517e-15	Rejet de H0
Monte Carlo	0.012847	0.95	1253	56	62.65	0.044693	0.769339	3.804216e-01	Non-rejet de H0

Test de Kupiec à 99 %

	VaR	Confiance	Observations	Exceptions	Exceptions attendues	Fréquence observée	Statistique LR	p-value	Décision
Paramétrique	0.012723	0.99	1253	58	12.53	0.046289	88.496270	0.000000	Rejet de H0
Historique	0.012299	0.99	1253	63	12.53	0.050279	104.632990	0.000000	Rejet de H0
Monte Carlo	0.018951	0.99	1253	17	12.53	0.013567	1.449104	0.228672	Non-rejet de H0

Les résultats du test de Kupiec confirment bien les analyses précédentes. Le modèle de Monte Carlo est considéré comme fiable pour le calcul de la VaR.

Backtesting de la VaR : exceptions observées et test de Kupiec



Etape 5 : Machine Learning : détection de régime de volatilité

Sommes-nous dans un régime de marché calme ou stressé ? (classification ML, utile pour ajuster dynamiquement les limites de risque)

- Ajout d'un feature variable `high_vol = 1` si la volatilité réalisée sur les 5 prochains jours dépasse le 75e percentile historique, sinon 0

```
In [14]:
"""
Pour chaque jour t, high_vol[t] = 1 si les 5 jours après t ont été agités (dans le quart le plus volatile de toute l'historique),
C'est la variable à prédire, on va l'utiliser pour entrainer le passé
"""

#volatilité réalisée sur les 5 prochains jours
vol_realisee_future = portfolio_return.rolling(5).std().shift(-5) #shift négatif pour indiquer le futur

#seuil = 75e percentile de la volatilité future, sur toute l'historique
seuil = vol_realisee_future.quantile(0.75)

#variable cible binaire
high_vol = (vol_realisee_future > seuil).astype(int)
```

- Construction des features

```
In [15]:
"""
Explication du drawdown, puisque c'est nouveau : cumulative reconstitue la valeur du portefeuille dans le temps (en partant de 1).
running_max retient le plus haut niveau jamais atteint jusqu'à ce jour.
e drawdown, c'est "de combien on a chuté par rapport à ce plus haut" – toujours négatif ou nul.
Un drawdown de -0.15 veut dire "on est 15% en dessous du pic historique récent".
C'est une feature utile parce qu'un marché déjà en forte baisse a souvent tendance à rester volatil.
"""

features = pd.DataFrame(index=portfolio_return.index)

#vol réalisée sur les 5 derniers jours (passé, pas de shift)
features["vol_5j"] = portfolio_return.rolling(window= 5).std()

#vol réalisée sur les 20 derniers jours
features["vol_5j"] = portfolio_return.rolling(window = 20).std()

#rendement moyen glissant sur 20 jours
features["rendement_moyen_glissant_20j"] = portfolio_return.rolling(window = 20).mean()

#drawdown courant : à quel point on est loin du plus haut historique récent
cumulative = (1 + portfolio_return).cumprod()
```

```
running_max = cumulative.cummax()
features["drawdown"] = (cumulative - running_max) / running_max
```

- régression logistique ET random forest training

```
In [16]:
X = features #features
y = high_vol #cible

data_m1 = pd.concat([X, y.rename("high_vol")], axis=1).dropna() #garanti que X et y restent parfaitement synchronisés Ligne par Li
data_m1

X = data_m1.drop(columns=["high_vol"])
y = data_m1["high_vol"]
```

- split chronologique (jamais aléatoire)

```
In [17]:
split_index = int(len(X) * 0.8)

X_train, X_test = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_test = y.iloc[:split_index], y.iloc[split_index:]
```

Pourquoi pas train_test_split avec mélange aléatoire : sur une série temporelle, mélanger reviendrait à entraîner le modèle en lui montrant parfois des jours de 2026 avant de tester sur des jours de 2022, ce qui n'a aucun sens (dans la réalité, tu ne connais jamais le futur au moment de l'entraînement). Le split chronologique simple (80% des premières dates pour l'entraînement, 20% des dernières pour le test) reproduit une vraie situation de prédiction.

- training

```
In [19]:
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler

# La régression logistique est sensible à l'échelle des features → on standardise
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

log_reg = LogisticRegression()
log_reg.fit(X_train_scaled, y_train)

# Le random forest n'a pas besoin de standardisation (il est basé sur des seuils, pas des distances)
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
```

Out [19]:

```
▼ RandomForestClassifier ⓘ ?
  ► Parameters
  ► Fitted attributes
```

- Evaluer le recall par classe, AUC, courbe ROC

```
In [20]:
from sklearn.metrics import classification_report, roc_auc_score, roc_curve
import matplotlib.pyplot as plt

for nom, modele, X_te in [("Régression logistique", log_reg, X_test_scaled), ("Random Forest", rf, X_test)]:
    y_pred = modele.predict(X_te)
```

```

y_proba = modele.predict_proba(X_te)[: , 1]

print(f"--- {nom} ---")
print(classification_report(y_test, y_pred, target_names=["Vol normale (0)", "Vol élevée (1)"]))
print(f"AUC : {roc_auc_score(y_test, y_proba):.3f}\n")

# Courbes ROC superposées
plt.figure(figsize=(8, 6))
for nom, modele, X_te in [("Régression logistique", log_reg, X_test_scaled), ("Random Forest", rf, X_test)]:
    y_proba = modele.predict_proba(X_te)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    plt.plot(fpr, tpr, label=f"{nom} (AUC={roc_auc_score(y_test, y_proba):.3f})")

plt.plot([0, 1], [0, 1], linestyle="--", color="gray", label="Hasard")
plt.xlabel("Taux de faux positifs")
plt.ylabel("Taux de vrais positifs")
plt.title("Courbe ROC : régression logistique vs random forest")
plt.legend()
plt.show()

```

Out [20]:

```

--- Régression logistique ---
              precision    recall  f1-score   support

Vol normale (0)       0.73      0.96      0.83      181
Vol élevée (1)        0.22      0.03      0.05       66

   accuracy              0.71      247
  macro avg       0.48      0.50      0.44      247
 weighted avg     0.60      0.71      0.62      247

```

AUC : 0.623

```

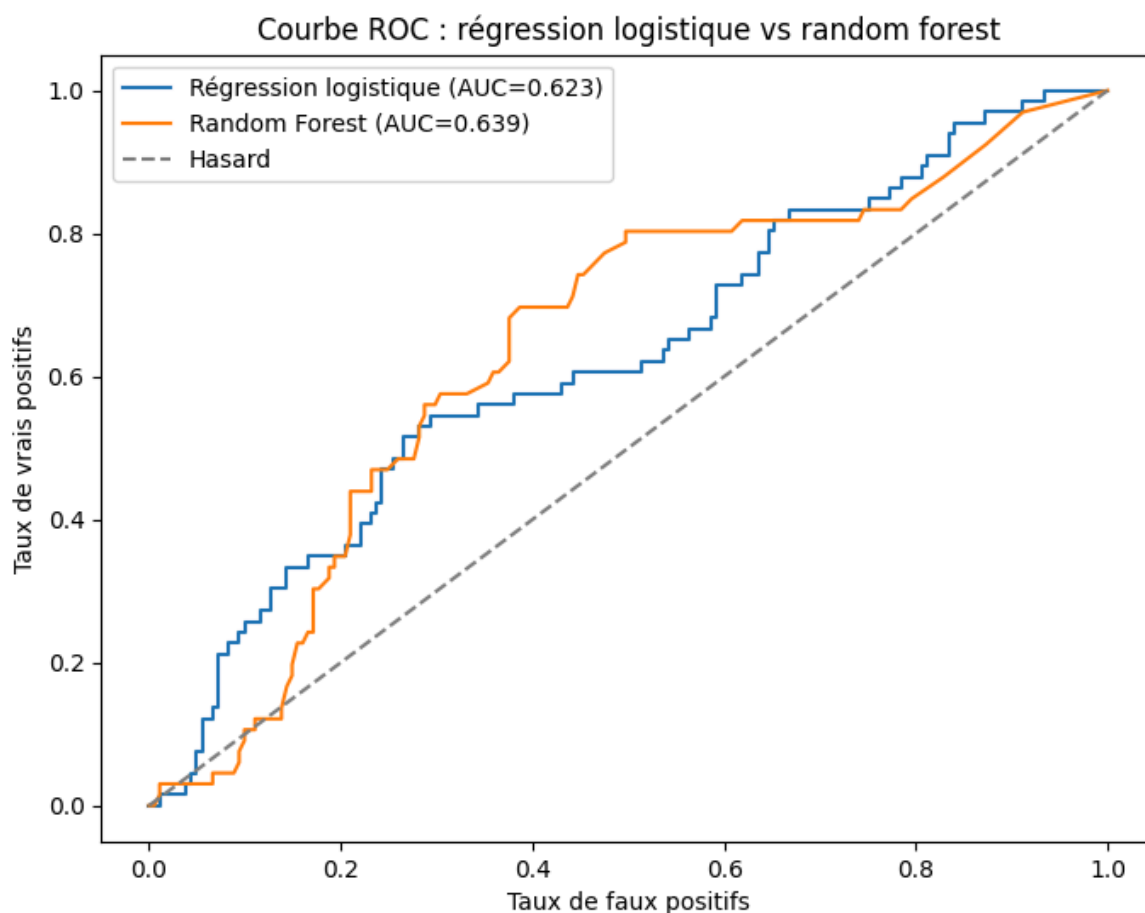
--- Random Forest ---
              precision    recall  f1-score   support

Vol normale (0)       0.73      0.88      0.80      181
Vol élevée (1)        0.27      0.12      0.17       66

   accuracy              0.68      247
  macro avg       0.50      0.50      0.48      247
 weighted avg     0.61      0.68      0.63      247

```

AUC : 0.639



Les recall des 2 modèles pour prédire un stress sont relativement bas (66 à 67 jours prédits sur 247 jours). Avec ces 4 features (volatilité glissante, rendement moyen, drawdown), les modèles ne parviennent pas à détecter significativement mieux qu'un modèle naïf le régime de volatilité à 5 jours. Ceci suggère soit que le signal prédictif de ces features est trop faible pour cet horizon, soit qu'il manque des variables informatives (le VIX, qu'on avait identifié comme piste, capture justement une dimension — le sentiment de marché — absente de ces 4 features basées uniquement sur les rendements du portefeuille lui-même). On pourrait améliorer le modèle en rajoutant le VIX.